Session 5: Functions in C++

1. Write a program to implement `swap()` by value and compare the result with reference-based swap.

example :

```
// Swap using Call by Value
void swapByValue(int a, int b)
{
   // TODO:
   // 1. Declare temporary variable
   // 2. Swap a and b
}

// Swap using Call by Reference
void swapByReference(________)
{
   // TODO:
   // 1. Declare temporary variable
   // 2. Swap a and b
}

int main()
{
   int num1 = 10;
   int num2 = 20;

   cout << "Before swapByValue: ";
   cout << num1 << " " << num2 << endl;

   swapByValue(num1, num2);

   cout << "After swapByValue: ";
   cout << num1 << " " << num2 << endl;

   cout << "\nBefore swapByReference: ";
   cout << num1 << " " << num2 << endl;

   swapByReference(_______________);
```

```
    cout << "After swapByReference: ";
    cout << num1 << " " << num2 << endl;

    return 0;
}
```

2. Write an inline function to calculate factorial and calculate power.

Sessions 6 & 7: Memory Management and Pointers

3.
Write a program to:
Ask the user for number of students.
Dynamically allocate an array of Student objects using new.
Each student contains:
rollNo
marks
Display:
All student records
Highest marks
Properly release deallocate memory.

4. Create a class named Laptop with the following specifications:
Class Requirements:
Private data members:
brandId
price
A parameterless constructor that:
Initializes brandId to 1
Initializes price to 50000
Displays the message:
"Laptop Constructor Called"
A destructor that displays the message:
"Laptop Destructor Called"
A member function display() that prints the laptop details.

5. Write your own implementations of `strlen()` and `strcpy()` without using `string.h`.

Session 8: OOP Concepts

6. Create a class BankAccount with private data members (accountNumber, accountHolderName, balance). Provide public member functions to deposit, withdraw, and display balance. Create an object in main() and demonstrate encapsulation by accessing data only through public functions i.e. deposit(), withdraw().

7. Create `Student` and `Teacher` class inside a namespace `college`. Write constructors, getter/setters, display() method in both classes. Create objects in main() and call methods.

Session 10: Inheritance & Polymorphism

8. Create class Programmer and class Teacher, both containing a function work(). Derive class ProgrammingTeacher from both classes. Create an object of ProgrammingTeacher and call work() to show ambiguity. Resolve the ambiguity using scope resolution operator. Add `cout` statements in constructors to confirm constructor calling sequence.

9. Create a base class Person with data members name and age and a member function display(). Derive two classes Student (with data member marks) and Teacher (with data member salary) from Person. Add a function in each derived class (study() for Student and teach() for Teacher). Add abstract method work() in Person and implement in both derived classes. Create Student and Teacher object in main() to demonstrate run-time Polymorphism.

Session 11: Polymorphism

10. Implement a `Matrix` class and overload `+`, `-`, and `*` operators.

11. Implement a singly linear linked list with operations `add_first()` and `add_last()` to demonstrate proper use of friend class.

Session 12: Virtual Functions and Abstract Class

12. Implement a `Product` (fields: title, price & virtual methods accept() & display()) class with derived classes `Book` (fields: pages) and `Tape` (fields: playtime). Create an array of 5 Product pointers and initialize it with Product & Tape objects as follows. Write a loop to display only pages and playtime values.

```
Product *arr = new Product*[5];
for(int i=0; i<5; i++) {
    cout << "1. Book, 2. Tape, Enter choice: ";
    cin >> choice;
    switch(choice) {
    case 1:
        arr[i] = new Book;
        arr[i]->accept();
        break;
    case 2:
        arr[i] = new Tape;
        arr[i]->accept();
        break;
    }
}
```

13. Create a `Payment` base class and derived classes `CreditCard`, `UPI`, and `Cash`. Write method `void makePayment(double amount)` in base class (choose virtual or abstract) and override in derived classes (Simulate payment with appropriate `cout` statements). Write a

static method `Payment* create(char *mode)` in a new `PaymentFactory` class. In main() get payment object via factory and test its makePayment().

Session 13: Exception Handling

14. Extend the BankAccount class (Assign 6) to include exception handling: in the deposit() and withdraw() functions, if the amount is negative, throw the amount as a double, and in the withdraw() function, if the amount is greater than the current balance, throw an error message as a string. In main(), use multiple catch blocks to handle these exceptions and display appropriate messages when they are caught.
15. Throw a custom exception class from `Student` (Assign 7) class methods to prevent negative marks.
16. What problem you see in the following code in exception stack unwinding? Resolve them using user-defined smart pointer.

```
try {
    Student *s = new Student;
    // accept info from user and set using setters
    s->display();
    delete s;
}
catch(...) {
    cout << "invalid input" << endl;
}
```

Session 14: File Handling in C++

17. Write a program to save and load `Student` object using file IO.

18. Write a generic class for `Array` to store user-defined data types using templates. Overload required operators. Prevent memory leakage.

Sessions 15: STL and RTTI

19. Create `list<int>` and store 10 random numbers. Do the following operations one after another.
- Display list in reverse order using iterator.
- Increment each number in the list by 5 using iterator.
- Display list in using const iterator.
- Sort the list using appropriate STL algorithm.
- Display modified list using default iterator.

20. Demonstrate ownership transfer with unique_ptr, shared ownership with shared_ptr, and non-owning reference with weak_ptr.